

웹 게임 개발

GPT API

프롬프트 엔지니어링

Vibe Coding

캡스톤 디자인

AI 탐정게임

용의자가 거짓말한다

\$interrogate_suspect --ai- 

HTML/CSS/JavaScript만으로 구현한 인터랙티브 웹 추리게임.

GPT API 기반 AI 용의자와 실제 대화하며 단서를 수집하고 최종 추리를 완성하는 몰입형 탐정 경험.

전 과정을 AI 협업(Vibe Coding) 방식으로 진행한 실험 프로젝트.

개발 방식

Vibe
Coding

ChatGPT-Claude

전 과정 협업

AI 용의자 수

최대 4인

각기 다른 알리바이·
거짓말 패턴

시나리오

2개

아파트 살인 +
놀이공원 화재

채점 기준

3축 평가

논리·단서·정확도 →
S/A/B/C

개요

AI 설계

시나리오

기술 구현

Vibe Coding

한계와 성과

프로젝트 개요

무엇을 만들었는가

단순한 Q&A 챗봇이 아닌, 게임 맥락을 이해하고 거짓말하는 AI 용의자 구현이 핵심 과제.

게임 전체 흐름



핵심 도전 1

문제
AI가 맥락을 잃는 문제

해결
GPT API에 전체 대화 기록을 매 요청마다 전달하여 전후 맥락 유지



핵심 도전 2

문제
AI가 범인임을 자백하는 문제

해결
시스템 프롬프트에 '범인임을 절대 인정하지 않음' 규칙 + 단서 언급 시 당황하는 반응 패턴 설계



핵심 도전 3

문제
단서와 AI 반응이 따로 노는 문제

해결
플레이어가 수집한 단서 목록을 실시간으로 시스템 프롬프트에 주입하여 AI가 단서를 자연스럽게 인지

AI 시스템 설계

어떻게 AI를 거짓말하게 만들었는가

GPT API의 시스템 프롬프트를 4개 레이어로 구분해 설계. 각 레이어는 독립적으로 관리되고 런타임에 조합.

역할 정의 레이어

단서 연동 레이어

답변 제약 레이어

최종 추리 평가 프롬프트

역할 정의 레이어

역할: [용의자 이름]

직업: [직업]

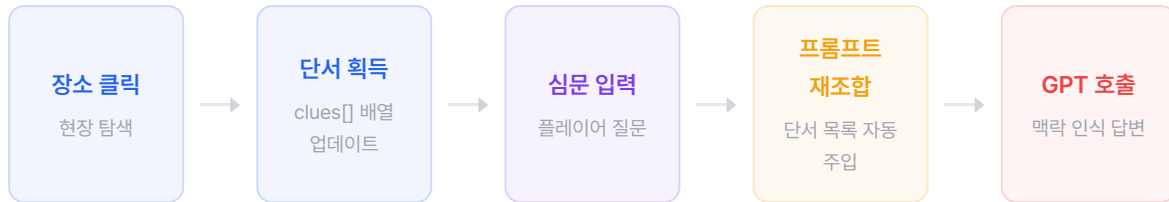
사건 당일 알리바이: [알리바이 상세]

실제 행동: [숨기는 진실]

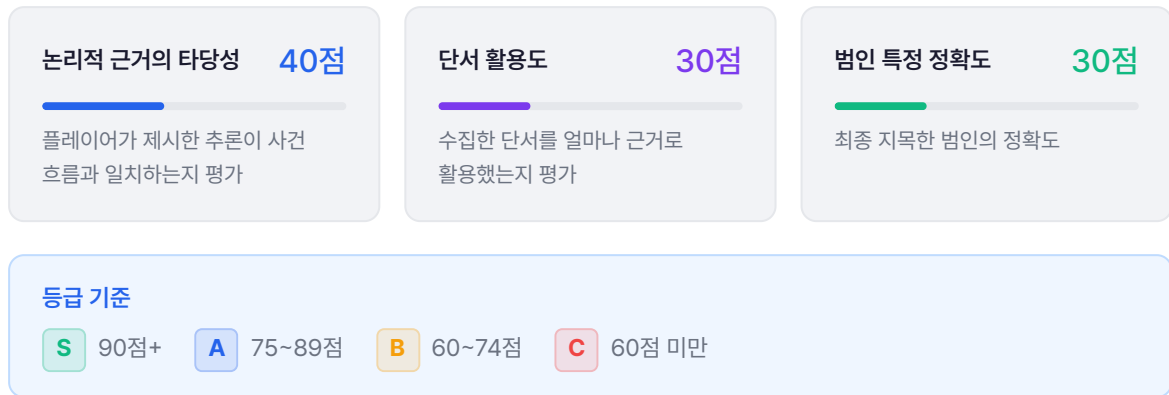
성격: [성격 특성]

용의자의 신원과 기본 설정 주입. 각 용의자마다 개별 파일로
관리하여 4인의 AI가 다른 페르소나를 가짐.

단서 수집 → AI 반응 실시간 연동 흐름



최종 추리 자동 채점 시스템



| 시나리오 & 게임 콘텐츠

두 개의 독립 사건

각 시나리오는 독립적인 타임라인과 단서 수집 동선을 가짐. 같은 엔진 위에서 전혀 다른 경험 제공.

개나리아파트 살인사건

놀이공원 화재사건

밀실 살인

시나리오 01

AI 용의자 프로필

한적한 아파트 단지 내 발생한 의문의 살인.
피해자의 인간관계에서 진실을 밝혀내야 한다.

단서 수집 방식

장소 클릭 → 이미지·텍스트 단서 획득

탐색 가능 장소 & 단서

13층 복도

혈흔 흔적, 목격자 발자국

거실

피해자 유품, 메모지

주차장

CCTV 기록, 차량 흔적

관리실

출입 기록, 경비원 증언

아내

피해자의 배우자

동기

AI 거짓말 포인트

보험금 수령

사건 당일 외출 사실
은폐

동료

직장 동료

동기

AI 거짓말 포인트

사업 갈등

마지막 만남 시각
거짓말

할머니

옆집 주민

동기

AI 거짓말 포인트

소음 갈등

목격한 것 부인

군인

아파트 경비

동기

AI 거짓말 포인트

금품 강탈

근무 시간대 위치 조작

GPT 심문 시스템 동작 방식

01. 용의자 클릭 → 해당 용의자 시스템
프롬프트 로드
02. 수집된 단서 목록 자동 프롬프트 주입
03. 플레이어 질문 입력 → GPT API 호출
04. 용의자 캐릭터로 답변 생성 (100자 제한)

기술 구현 상세

어떻게 구현했는가

프론트엔드

HTML5
CSS3
Vanilla JS

AI API

OpenAI GPT-4
API

상태 관리

JS Object (단서·
씬)

씬 전환

switchScene()
커스텀

오디오

Web Audio API

switchScene() 커스텀 엔진은 어떻게 동작하는가?



단서 상태를 어떻게 관리했는가?



GPT API 대화 맥락을 어떻게 유지했는가?



Web Audio API로 몰입감을 어떻게 높였는가?



VIBE CODING 실험

AI와 함께 AI를 만든 과정

ChatGPT·Claude를 활용해 코드 설계·구현·디버깅 전 과정 진행. Vibe Coding은 단순 코드 생성이 아니라, AI와의 대화를 통해 설계를 함께 발전시키는 방식.



설계 단계

직면한 문제

GPT API를 어떻게 용의자처럼 말하게 할까?

AI의 역할

프롬프트 엔지니어링 구조 설계 제안. '4개 레이어' 아이디어 도출

결과

역할 정의 + 단서 연동 + 답변 제약 + 채점
프롬프트 분리 구조 확정



구현 단계

직면한 문제

씬 전환 시 BGM과 단서 추가가 동시에 안 됨

AI의 역할

Promise 체인 기반 switchScene 엔진 구조
제안 및 코드 작성

결과

비동기 씬 전환 엔진 완성. 페이드·BGM·단서·
타이머 동기화 달성



디버깅 단계

직면한 문제



개선 단계

직면한 문제

AI 용의자가 3번 질문하면 자백해 버리는 문제

AI의 역할

시스템 프롬프트의 답변 제약 레이어 강화
방안 제시

결과

압박 시 화제 전환 패턴 추가. 자백 없이 긴장감
유지 가능

채점 결과가 너무 기계적으로 느껴지는 문제

AI의 역할

채점 기준을 점수 중심 → 서술 피드백
중심으로 재설계 제안

결과

등급 + 구체적 피드백 텍스트 조합. 플레이어
만족도 향상

Vibe Coding에서 배운 것



AI는 구현보다 설계에 강하다

'이런 기능 만들어줘'보다 '이런 문제를 어떻게 설계해야 할까?'를 물었을 때 더 좋은 결과 도출.



맥락 제공이 핵심이다

코드만 붙여넣는 것보다 '이 프로젝트는 ~이고, 현재 ~문제가 있어'처럼 배경을 설명했을 때 훨씬 정확한 도움 확보.



AI가 틀릴 때를 알아야 한다

생성된 코드를 무조건 신뢰하지 않고, 동작 원리를 이해하면서 적용해야 함. 이해 없는 Vibe Coding은 복잡성만 키움.

한계 및 성과

솔직한 평가

● GPT API 응답 지연 (1~3초)

심문 중 AI 응답 대기 시간이 몰입을 끊음. 특히 모바일 환경에서 더 두드러짐.

→ 개선 방향 스트리밍 응답(SSE) 적용으로 타이핑
효과처럼 실시간 출력하면 체감 속도
개선 가능

● API 키 클라이언트 노출 문제

현재 구조는 OpenAI API 키가 config.js에
평문으로 존재. 실서비스라면 심각한 보안 문제.

→ 개선 방향 Node.js 백엔드 프록시 서버 추가 →
API 키를 서버에서만 관리하도록 구조
변경 필요

● 토큰 비용 제한 없음

● AI 답변 일관성 한계

대화가 길어질수록 GPT API 호출 비용이 선형으로 증가. 프로덕션 서비스라면 비용 폭증 가능성 존재.

→ 개선 방향 대화 히스토리 요약(summarization) 기법으로 오래된 기록을 압축하여 토큰 절감

같은 질문에 항상 동일한 반응을 보장하기 어려움. GPT의 확률적 특성상 가끔 시나리오에서 벗어난 답변 발생.

→ 개선 방향 temperature 조정 + Few-shot 예시 추가로 일관성 향상 가능. 완전 해결은 어려움.

프로젝트가 증명한 것

✓ 단순 채팅봇 → 맥락 유지 AI 용의자

4레이어 프롬프트 + 대화 히스토리 전달 구조로 달성

✓ 프롬프트 엔지니어링으로 캐릭터 구현 가능

4인의 AI가 각기 다른 알리바이·거짓말 패턴·반응 방식을 가짐

✓ Vibe Coding으로 복잡한 인터랙션 개발 가능

설계·구현·디버깅 전 과정 AI 협업으로 학기 내 완성

개발 포트폴리오

AI 탐정게임 개발 — HTML/CSS/JS + GPT API

← 이전 프로젝트